

摘要：

MiniMax M3旗舰模型聚焦Coding与Agent能力，具备百万级长上下文和原生多模态，支持自主执行复杂长任务（如论文复现、CUDA优化）。实测中，其代码交付高效，能高精度复刻网页并推断交互逻辑。

一款开源模型，能否同时拥有顶级编程能力、超长上下文理解能力和原生多模态能力？

这几乎就是 Agent 的全部意涵。而我们提出这个问题，是因为从 OpenClaw 时代开始，一家公司就已经无法仅仅凭借在模型上的投入，证明自己是一家押注未来的公司。胜负全在 Agent。

MiniMax M3 似乎也意识到了这一点。

作为 MiniMax 的最新款旗舰模型，M3 重点强化了 Coding 与 Agent 能力。相比传统代码模型的“把代码写出来”，它更强调长期规划、多轮协作和自主执行复杂任务的能力。

通俗地说，这些能力共同指向一个目标，那就是让模型独立学习几十万字的资料、持续工作数小时、调用工具、编写代码，并最终交付一个真正可用的结果。这成为了同步推出的 MiniMax Code 产品的核心技术基础。

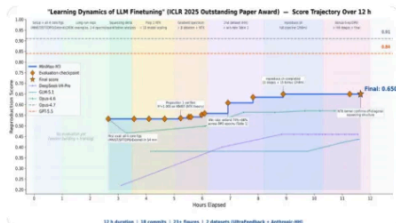
那么衍生出来的问题是，当 Claude Code 已经成为开发者最认可的 Agent 工具之一，M3 的能力，又是否足以支撑 MiniMax 建立一个自己的，真正有竞争力的 Agent 生态？

12 小时自主工作，你说的长任务有多长？

Coding 能力的进化，已经不仅仅是写代码了。

如果只把 MiniMax M3 当成一个更擅长写代码的模型，会严重低估此次发布的重点。M3 更值得拿出来讨论的，是它在长任务、长上下文和 Agentic 工作流上的能力。

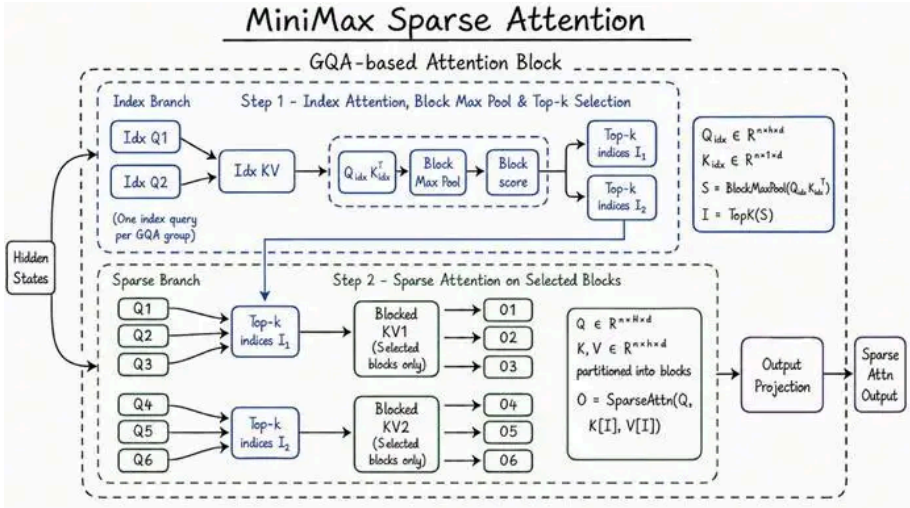
官方给出的两个案例很能说明这一点。一个是 M3 用接近 12 小时自主复现 ICLR 论文，另一个是用约 24 小时、147 轮迭代完成 CUDA Kernel 优化。这两个例子本质上都是典型的长链路任务，模型需要理解目标、拆解步骤、不断检查中间结果，并在失败之后继续调整。



论文复现：12 小时自主完成 ICLR 杰出论文

我们丢给 M3 一篇 ICLR 2025 杰出论文 — Learning Dynamics of LLM Finetuning，让它独立复现。M3 连续运行近 12 小时，全程自主产出 18 次 commit 与 23 条实验图表，成功跑通核心实验，多模态看懂论文里的图表公式，长上下文保证论文 + 代码 + 实验日志一次性进窗口。编程 + Agent 能力驱动长线程执行。

从模型架构上看，MiniMax M3 的 1M token 上下文和 MSA 稀疏注意力架构，就是为这类场景服务的。长上下文的意义不只是能塞进更多文本，更重要的是降低长任务中的信息断裂。比如一个真实代码仓库、一个复杂需求文档、一组历史修改记录，这些真实需求都不是几千 token 就能讲清楚的。如果模型每次只能看到局部，就很容易出现“前面答得对，后面改崩了”的情况。而更长的上下文窗口，则给了模型跨文件、跨阶段理解任务的可能。



不过必须澄清的是，官方宣传的 1M 上下文，并不等于当前所有开发者都能无门槛、稳定地使用完整的 1M 上下文能力。模型页虽然写明“支持最高 1M，保证至少 512K”，但按量计费页进一步说明，超过 512K 的输入能力在发布初期属于限时、限量供应，需要联系销售开通。

长上下文能力确实是这次 M3 发布的核心亮点，但在真实任务中，它更适合被理解成一种“能力上限”，而不是一个已经对所有用户完全开放的默认规格。

创业模拟器，M3 与 Sonnet 4.6 的直接竞技

为了测试 M3 的代码交付能力，我设计了一个相对完整的小项目，让模型从零实现一个“创业模拟器”小游戏。同样接受这项考验的，还有 Claude Sonnet 4.6。

请从零开发一个 AI 创业模拟器 Web App。

要求：

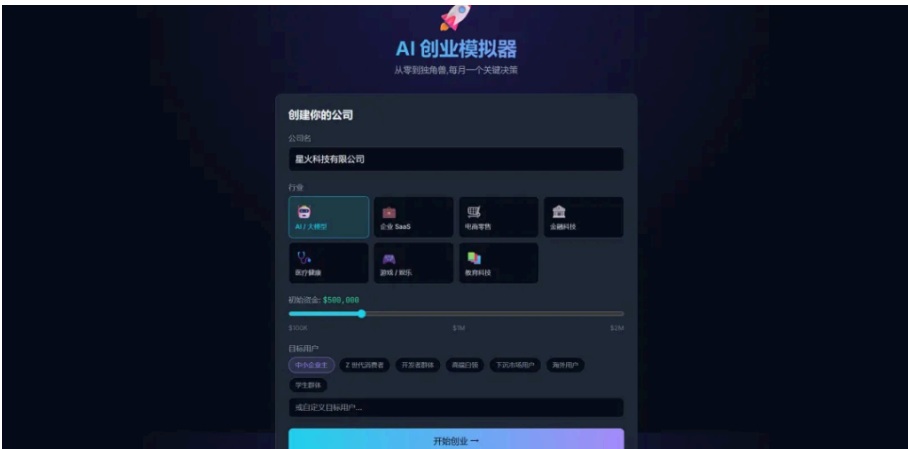
1. 用户可以创建一家初创公司，输入公司名、行业、初始资金、目标用户。
2. 游戏采用回合制，每一轮代表一个月。
3. 用户每轮可以选择 3 个经营决策，例如产品开发、市场推广、招聘、融资、降本、用户调研。
4. AI 根据当前公司状态和用户决策生成月度报告。
5. 页面需要展示资金、用户数、收入、团队士气、产品完成度、市场热度、竞争压力。
6. 每轮结束后更新这些指标。
7. 需要有成功和失败结局。
8. 使用 React + Tailwind 实现，界面要像一个现代化创业经营游戏。
9. AI 接口可以先用 mock 数据，但代码结构要方便之后接入真实 LLM API。
10. 请保证项目可以运行，并提供启动方式。

提示词并不复杂，但这项任务其实很适合测试 Coding Agent 的综合能力。因为它同时考验需求理解、状态管理、UI 设计、数值系统和平衡性。用户在游戏中扮演创业者，每一轮需要决定做什么产品、招什么人、怎么定价、要不要融资、如何营销，AI 则根据这些决策反馈用户增长、现金流、团队士气、市场反应和竞争压力。

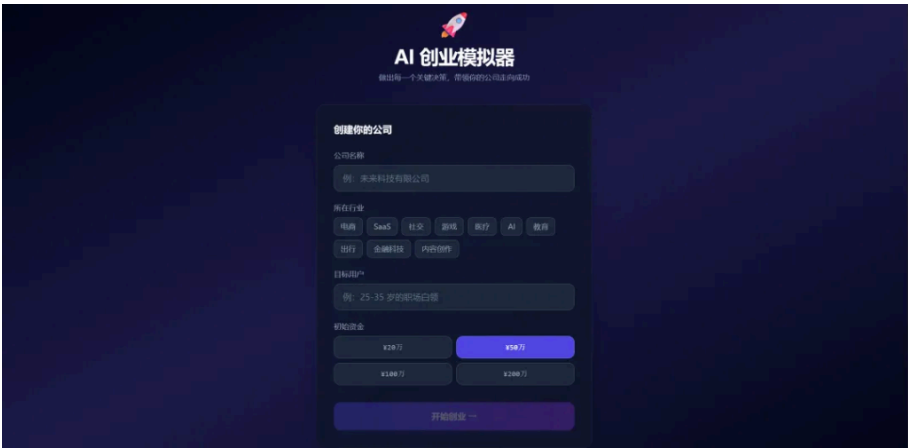
具体来说，真正的难点主要包括三个维度：

- 状态管理：小游戏一旦进入多轮决策，就容易出现页面刷新后数据丢失、上一轮数据覆盖下一轮、历史记录无法回看、进度条超过 100% 之类的问题。**甚至游戏只是这些问题的高发场景，类似的需求，在很多软件开发任务中都可以看到。**
- UI 表现：很多模型生成的“游戏”其实只是一个表单加几个按钮，功能能跑，但一眼看过去就有股“塑料感”。
- 数值平衡：这是最难的一环，数值设计不当很容易出现一两轮游戏之后现金流爆炸、用户数异常增长、游戏迅速失控的问题，最终影响可玩性。**什么样的数值设计可以说是平衡？这需要模型在复杂任务拆解之外，更有一层对游戏的审美和品味。**

M3 用大约 11 分钟完成了程序编写和代码检查。最终生成的小游戏可以正常运行，界面简洁，并且带有一定动画效果。更重要的是，它基本处理好了前面提到的几个核心难点，公司数据展示清晰，历史记录可以回看，游戏进度和经营指标也没有明显混乱。



作为对比的是，Sonnet 4.6 完成同一任务大约用了 19 分钟。它同样让游戏正常跑了起来，还在内容设计上增加了一点小巧思。比如加入突发事件，让游戏难度和不确定性更强，游戏性确实更高。



这是个很有意思的差异。

基于 M3 的 MiniMax Code 更像是一个执行力很强的工程师 Agent，它会非常忠实地围绕你的 prompt 做交付。优势也在这里，动作快，完成度高，指令给过去，他会围绕最终产物，把页面、逻辑、状态和基础交互一起搭出来。

而基于 Sonnet 4.6 的 Claude Code 则更像一个会主动补充产品想法的合作者，它可能会在需求之外加入一些额外的设计。

这两种风格没有绝对好坏。**如果你的需求非常明确，希望模型严格按照指令快速完成，M3 的表现会非常令人舒适，毕竟谁不想要一个指哪打哪的员工。**但如果你期待模型主动补完产品创意、增强玩法、提出更多可能性，Sonnet 4.6 目前在创造性扩展上仍然更有优势。

看图写前端：原生多模态能力实测

相比于长任务和 Coding 能力，多模态可能是 MiniMax M3 身上最容易被低估的一项能力。

很多模型宣传自己支持图片输入，但实际体验下来，往往停留在“看图说话”的阶段，能够描述页面里有哪些元素，却很难将这些视觉信息进一步转化为可运行的代码。而 M3 此次给我的最大惊喜恰恰在于，它展现出了从视觉理解到工程交付的完整链路能力。

为了测试这一点，我选择了一个非常直接的场景，将 MiniMax 自己的官网首页作为测试对象。我向 M3 提供了两张首页截图，并要求它使用 React 与 Tailwind CSS 对页面进行复刻。

根据这张网页截图，使用 React + Tailwind CSS 完整复刻页面。

要求：

1. 尽可能还原原页面的：

- 整体布局
- 字体层级
- 卡片设计
- 配色方案
- 间距与留白
- 按钮样式

2. 页面必须响应式，适配：

- Desktop
- Tablet



- Mobile

3. 识别并还原：

- Hero Section

- 导航栏

- Feature Cards

- CTA Button

- Banner

- Footer

4. 使用组件化结构：

- Navbar.tsx

- Hero.tsx

- FeatureCard.tsx

- Footer.tsx

5. 不要使用占位符代码。

6. 输出完整可运行代码。

让生成页面与截图视觉相似度达到 90% 以上。

之所以选择官网首页，是因为这类营销页面往往包含大量视觉设计细节：导航栏、卡片模块、渐变背景、按钮样式、信息层级以及复杂的页面布局。对于模型而言，这不仅是在识别图片中的文字，更是在理解整个页面背后的设计逻辑。

最终结果让我有些意外。

首先是页面结构的还原度。

仅凭两张截图，M3 对首页整体布局的复刻已经达到了极高的水平。导航栏、Hero 区域、功能介绍模块以及各个内容板块之间的层级关系都被准确识别出来，页面整体结构与原网页几乎保持一致。



如果只从宏观布局来看，几乎已经到了以假乱真的程度。剩下的差异主要集中在一些字体间距、元素对齐方式等细节层面。但就是把这些不一样的局部画面单独截图出来，你都得回忆一下，MiniMax 那个正版的官网画面是不是就长这样。

更有意思的是，M3 并没有机械地“照抄截图”。

由于测试时我只提供了首页部分内容，理论上模型无法得知页面下半部分应该如何设计。但在实际生成过程中，M3 并没有简单地留下空白，而是主动分析了官网整体的视觉风格和配色特点，自行后续页面补充了若干风格一致的内容模块。虽然这些内容并不完全对应真实官网，但无论是配色方案还是设计语言，都与原页面保持了高度一致，整体看起来并不会让人产生明显的割裂感。

这一点其实非常重要。因为它说明模型并不仅仅是在做 OCR 或者截图复刻，而是在尝试理解页面背后的设计规律，并利用这种理解完成合理推断。

除了视觉层面的还原之外，M3 对交互元素的识别也给我留下了不错的印象。

在生成结果中，模型正确识别出了导航栏、按钮等交互式组件，并为这些元素赋予了实际功能，例如导航栏中的菜单项可以直接跳转到对应内容区域，按钮组件也被正确实现为可点击元素。

更进一步，M3 还主动为页面补充了交互动效。当鼠标悬停在按钮上时，页面会出现过渡动画与视觉反馈。这些效果并没有出现在我的提示词中，而是模型根据现代 Web 产品的设计习惯自行加入的细节。

综合来说，M3 展现出了相当强的竞争力。它不仅能够理解网页截图中的结构信息，还能识别交互逻辑、推断缺失内容，并最终生成一个能够运行、能够交互、视觉风格高度一致的前端页面。

当然，它并非没有不足。页面中仍然存在一些排版细节上的偏差，但考虑到整个过程几乎完全由模型自主完成，并且输入仅仅是两张截图，这样的结果已经远超最初的预期。

价格也是生产力

价格是大模型竞争中最现实的话题。过去一年，AI 行业几乎经历了一轮全面价格战，DeepSeek 用极低的 API 成本掀翻市场，OpenAI、Anthropic 和 Google 持续提升模型能力的同时也在不断调整定价策略。

从官方定位来看，M3 主打的是 Frontier Coding、Agent、多模态与百万级上下文能力。这首先决定了它的竞争对手，不是那些用于智能客服、会议纪要的中端模型，而是当前行业最前沿的一批旗舰模型，比如 Claude Opus 4.8、GPT-5.5、Gemini 2.5 Pro、GLM-5.2 以及 DeepSeek V4-Pro 等。

直接看价格，目前 Claude Opus 4.8 的 API 价格为输入 5 美元/百万 Token、输出 25 美元/百万 Token。GPT-5.5 为输入 5 美元、输出 30 美元。DeepSeek V4-Pro 在最新降价后为输入 0.435 美元、输出 0.87 美元。相比之下，MiniMax M3 官方价格为输入 0.6 美元、输出 2.4 美元。

如果以 Claude Opus 4.8 为基准，M3 的输入成本仅约为其 12%，输出成本不到 10%，即便面对 OpenAI 最新的 GPT-5.5，M3 的调用成本也只有其十分之一左右。换句话说，在同样消耗

100 万输入 Token 和 100 万输出 Token 的情况下，使用 GPT-5.5 需要 35 美元，使用 Claude Opus 4.8 需要30美元，而 M3 仅需 3 美元。

对于用量不大的普通用户来说，这种差异尚不明显，但如果你是已经习惯了每天靠大量 Agent 处理长文档、批量生成代码或者构建 AI 应用的开发者，成本差距则会被迅速放大。**假设一个项目每月消耗 1000 万输入 Token 和 1000 万输出 Token，使用 Claude Opus 4.7 的成本约为 300 美元，而使用 M3 仅需 30 美元左右。在保持接近旗舰模型能力的前提下，十倍左右的成本优势已经足以影响技术选型。**

当然，价格从来不能脱离能力讨论。

如果 M3 只有廉价可圈可点，那么这样的比较并没有意义。但有意思的是，在 MiniMax 公布的多项评测中，M3 瞄准的正是 Claude Opus 4.7、GPT-5.5 和 Gemini 旗舰模型所在的竞争区间。在 SWE-Bench Pro 等代码能力测试中，M3 已经超过 GPT-5.5 与 Gemini 旗舰模型，接近 Claude Opus 4.7，在长任务 Agent 场景下，官方展示的论文复现和 CUDA 优化案例甚至能够持续自主运行十几个小时以上。

更关键的是，M3 并非依靠阉割能力来换取低价格。如前所述，这份价格背后是 100 万 Token 上下文窗口、原生多模态架构以及 Agent 工作流能力，而 Claude、GPT 和 Gemini 恰恰也是沿着同样的方向演进。换句话说，M3 试图参与竞争的并不是“便宜模型市场”，而是最昂贵、也是技术含量最高的旗舰模型市场。

因此，如果只看绝对价格，DeepSeek V4-Pro 仍然是目前最激进的价格屠夫。但如果同时考虑 Coding、Agent、多模态和超长上下文这些旗舰能力，那么 M3 可能是目前整个市场里最具冲击力的性价比选手之一。

MiniMax Code 的野心

几项测试下来，MiniMax M3 给我的感受是，它已经可以位列国产模型里最值得关注的 Coding / Agentic 底座模型之一，尤其在长任务、长上下文、多模态输入和代码交付方面，展现出了很强的竞争力。

注意底座模型这个定位，此前城头变幻大王旗的各种 Benchmark 一度让性能水平成为衡量大模型的唯一角度。但是当我们讨论 Agent，讨论落地，更现实的维度是可用性。MiniMax M3 看起来无意再去挑战“最强模型”的地位，而是在尝试成为 Agent 时代最具性价比的基础设施。

这是一条更清晰的路径。随着 Claude Code、Codex 等 Agent 产品逐渐成为开发者的主要入口，模型越来越回归其原本的角色，即一种底层能力。对于开发者而言，一个模型是否能完成长任务、调用工具、理解图像并控制成本，远比单纯跑分更重要。

从这个角度看，MiniMax 的策略相当清晰。M3 在长上下文、多模态和 Coding 能力上稳稳跻身第一梯队，同时又以远低于 GPT、Claude 的价格，将这些能力带到更多真实工作流之中。

真实工作流，这也是 MiniMax Code 此刻问世的原因。

卖 Token 的商业模式昙花一现，事实是开发者用脚投票的时候毫不犹豫，API 供应商越来越容易被替代。在这种情况下，模型能力领先 3 个月，不代表就有 3 个月的商业优势。这迫使模型厂商追问，为什么要把最有议价权的入口拱手让人？



此外今天生产级的 Agent，已经是一种高度集成的系统工程能力。一个复杂任务的交付水平，只有部分取决于模型，还有部分取决于 Agent Runtime。如果找对测评角度，每家都有“SOTA”模型，那么执行层的争夺将成为新的竞争焦点之一。

所以 MiniMax Code 是一个写代码的软件吗？

这仍然是 IDE 的视角。它实际上是模型、代码库上下文、工具调用、执行环境、工作流编排，所有决定 Agent 最终效果的东西都在这里了。有了这些，MiniMax 才有了争夺开发者工作流入口的资格。

本文来源：[雷锋网](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码

免费领取

限免

85 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

1 条评论

见闻用户

17小时前 中国北京

出rust python 专项

1

回复